

Name : K. Tharunika
Reg no : 192525276
Date : 04-08-2026

Course name : computer
Architecture
Course code : CSA-0315

Co-2&AT-2
Debugging

Q1. Debugging signed Integer Addition using 2's Complement.

Step 1: Binary Representation

$$+120 = 01111000$$

$$+50 = 00110010$$

Performing addition:

$$\begin{array}{r} + 01111000 \\ 00110010 \\ \hline 10101010 \end{array}$$

Binary Result =

$$10101010$$

In 2's complement,

$$10101010 = -86$$

A 8-bit signed integer
can be represent only
-128 (or) +127.

Hence Overflow
Occurs

Overflow detection:

overflow in signed addition when:

- * Two positive numbers produce a negative result
- * Two negative numbers produce a positive result

Here:

- operand signs = positive
- Result sign = negative

Source of error:

The processor stores only 8 bits, since 170 exceeds +127, the MSB changes from 0 to 1 causing negative result

Debugging steps:

Optimized solution

- | | |
|----------------------------|--|
| ① verify binary conversion | ① Add overflow detection |
| ② check carry | ② use 16 bit integers |
| ③ compare operand signs | ③ instead of 8 bit |
| ④ detect overflow | ④ use compiler overflow flag |
| ⑤ prevent incorrect output | ⑤ use display "overflow error". If overflow occurs |

Conclusion:

The incorrect answer occurs between the result exceeds the representable range of 8 bit signed number overflow detection & larger data types solve problem

Q2.

Ripple carry Adder Delay

operation: Each Full Adder waits for carry from previous stage

$cin \rightarrow FA0 \rightarrow carry_0$

↓

$FA1 \rightarrow carry_1$

↓

$FA2 \rightarrow carry_2$

↓

$FA3 \rightarrow count$

The final carry is available only after stages complete

Performance Bottleneck:

Delay increases with number of bits

Example:

① N -bit adder

Delay = $N \times$ Full Adder Delay

② 32-bit adder

Delay = $32 \times$ Full Adder Delay

Thus ripple carry is low

Carry Look-Ahead Adder:

CLA computes carries simultaneously

Generate: $G_i = A_i \times B_i$ Propagate: $P_i = A_i + B_i$

Carry equations: $C_1 + G_0 = P_0 C_0$

All carries are $C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$

Computed in parallel $C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$

Advantages:

Optimisation:

* Much faster

* Lower delay

* Better high speed processor

* Efficient for large bit width

① Replace carry adder

Ripple with carry look ahead adder

② Use parallel carry generation.

Conclusion: Ripple carry Adder suffers from cumulative carry delay. CLA delay significantly by calculating carry's simultaneously.

Q3. Booth's multiplication Algorithm debugging:

Booth Algorithm steps:

Registers: $A = 0000$

$Q = \text{multiplier}$

$M = \text{multiplicand}$

$Q-1 = 0$

Repeat it for n bits

Bit pair rules:

$Q_0 Q_{-1}$

00 $\rightarrow$ No operation

11 $\rightarrow$ No operation

01 $\rightarrow$ Add M

10 $\rightarrow$ Subtract M

After every operation, Arithmetic Right Shift

possible errors:

1. incorrect bit pair checking
2. logical shift used instead of arithmetic shift
3. sign bit not extended
4. wrong subtraction
5. Incorrect loop count

Debugging process

check:

- Initial register values
- Bit pair
- Add subtract operation
- Arithmetic shift
- Sign extension

Correction:

- * Arithmetic right shift
- * Proper sign extension
- * Correct subtraction using 2's complement
- * Exact number of iteration

Advantages of Correct Booth Algorithm:

- ① Handles signed multiplication
- ② Reduces number of additions
- ③ Faster multiplications.

Conclusion; most Booth application errors arise from improper sign extension, incorrect shifts, or wrong bit pair evaluation. correct arithmetic shift & sign handling ensure accurate results.

Q4. Restoring Division VS Non restoring Division;

Restoring Division

steps:

- ① shift left
- ② subtract divisor
- ③ if result negative:
 - Restore previous value
- ④ set Quotient bit
- ⑤ Repeat

problem:

Whenever subtraction become negative, restoration is performed. Extra operation increase execution time

Bottle neck:

Example Subtract

↓

Negative?

↓

Restore

↓

Continue

Repeated restoration wastes clock cycles

Non Restoring Division:

Instead of storing

Negative-remainder

↓

Next cycle perform addition

Positive remainder

↓

Next cycle performs subtraction

No restoration step is needed

Advantages:

- * Fewer arithmetic operations
- * Reduced clock cycles
- * Faster execution
- * Better processor performance.

Comparison:

Restoring

Performs restoring

More clock cycles

Slower

Lower efficiency

Non Restoring

No restoration

Fewer cycles

Faster

Higher efficiency

Optimization:

Replacing Restoring division. This minimize unnecessary operations

Conclusion:

Non-restoring division improves execution speed by eliminating restoration steps, making it more efficient for embedded systems

Q5. IEEE 754 Floating point Debugging

IEEE 754 single prediction:

32 bits

Sign = 1 bit

exponent = 8 bits

Mantissa = 23 bits

Cause of error:

example:- $1.0001000 + 0.000001$

other source of error:

- ① Exponent alignment
- ② Rounding
- ③ limited mantissa prediction
- ④ loss of significant digits
- ⑤ normalization.

Debugging

Check

- exponent alignment
- mantissa shifting
- normalization
- Rounding mode
- Final binary result

Optimization Techniques:

- ① Double precision:
 - 64 bit floating point
 - more exponent bits
 - larger mantissa
 - higher accuracy
- ② Kahan summation:
 - Reduces cumulative rounding errors while adding many numbers.
- ③ Higher precision Arithmetic:
 - Use extended precision whenever possible
- ④ Stable Algorithms: Avoid subtracting nearly equal values, rearrange computations to reduce precision loss.

Conclusion: IEEE 754 single precision loss accuracy because of exponent alignment and limited mantissa size. Using double precision, Kahan summation & numerically stable algorithms improves significantly floating point accuracy.